

Finding & Using Public Exploits

Unit 13 — Module 3: Exploitation

From "what version is it running?" to "here's a known exploit for it" — and how to use that exploit **safely and responsibly**.

Learning Objectives

By the end of this unit you can:

- **Trace** the workflow: enumerate → find service/version → find the known vulnerability → find a matching exploit.
- **Explain** what a **CVE** is and look one up on **NVD** and **Exploit-DB**.
- **Use** `searchsploit` to find an exploit matching a service and version.
- **Read** an exploit and identify what it targets, what it does, and anything dangerous.
- **Match** and **lightly adapt** an exploit (IP, port, parameter) for an authorized lab target.
- **Run** it against the **isolated lab** and document CVE → exploit → result.
- **Recommend** patching as the primary defense.

The workflow we're connecting

A version number is the **key** that unlocks everything.

1. **Enumerate** the target (Unit 8) → find open ports.
2. **Identify** the service **and its version** → e.g., `vsftpd 2.3.4`.
3. **Look up** known vulnerabilities for that version → a **CVE**.
4. **Find** a public exploit that matches → `searchsploit`, Exploit-DB.

No version, no search. The exact version is what makes the match possible.

CVE, NVD, and CVSS

Term	What it is
CVE	A unique ID for one known vulnerability, e.g. <code>CVE-2011-2523</code>
NVD	National Vulnerability Database — U.S. gov database with details + scores
CVSS	A 0–10 score showing how severe the vulnerability is

A scan gives you the version; **NVD** tells you if that version is dangerous and how badly.

Exploit-DB and searchsploit

- **Exploit-DB** — a giant public archive of exploits and PoCs, run by OffSec.
- **searchsploit** — searches a **local copy** of Exploit-DB from the command line (ships on Kali).

```
searchsploit -u           # update the local copy first
searchsploit vsftpd 2.3.4  # search by product + version
searchsploit -x <path>    # view an exploit
searchsploit -m <path>    # copy it to your folder
```

A polished exploit vs. a rough **proof of concept (PoC)** — read the columns and know which you've got.

Matching an exploit to your target

Before you trust a candidate, confirm it actually fits:

- **Service** — same software?
- **Version** — same or compatible version?
- **Platform / OS** — Linux vs. Windows?
- **Architecture** — x86 / x64, if it matters.

Grabbing an exploit for the **wrong** version or platform is the #1 reason a beginner thinks "exploits don't work."



Read the code before you run it

Never run untrusted code blindly.

A random exploit off the internet can be **booby-trapped** to attack *you*, or quietly do something destructive. You read **every line** first — for your safety and as professional diligence.

Read-before-you-run checklist

For any exploit, answer **every** question first:

- What **language / type** is it? (Python, Ruby, C, Metasploit module, manual steps?)
- What **service / version / platform** does it target?
- **Where** does it connect — which line holds the **IP** and **port**?
- What does the **payload** do? (open a shell? add a user?)
- Are there **comments** explaining it?
- Anything **destructive or suspicious**? (deletes files, phones home, code you can't explain)
- **Verdict:** safe to run in the lab? Why?

If anything looks suspicious — **do not run it.** Flag it for the instructor.

Lightly "fixing" an exploit

Edit only what's needed to point an **understood** exploit at your **authorized** target:

```
# change the hard-coded target IP to your lab box
# change the port if your scan found a different one
# set a listener if it returns a reverse shell:
nc -lvnp 4444
```

- Keep it **light** — IP, port, a parameter.
- Write down **each line you changed** and **what it does**.
- Full exploit development is out of scope (that's a rabbit hole).

Example: a clean version → shell story

The **vsftpd 2.3.4 backdoor** (`CVE-2011-2523`) on Metasploitable:

```
nmap -sV <target-IP>          # 21/tcp open ftp vsftpd 2.3.4
searchsploit vsftpd 2.3.4      # finds the backdoor exploit
# run the understood exploit against the LAB box only
whoami                        # -> root
id                             # -> uid=0
```

A username ending in `:)` triggers a hidden backdoor that opens a **root shell** on port 6200.

Lab-only. This box lives on an isolated network and exists to be attacked.

10



Defenses: patching

Every public exploit has the same boring, powerful defense:

- **Patch / update** — install the version that removes the vulnerability.
- **Remove end-of-life software** — unsupported = unpatched = easy target.
- **Patch management** — a process to track and apply updates fast.

Old, unpatched services are the easiest targets in the world. A finding without a remediation is **incomplete** — attacks are always paired with defenses.

Ethics & Authorization

Downloading an exploit is legal. **Running** it against a system you don't own — even "just a PoC," even "just to see" — is a crime under the **CFAA** and state law.

Targets here are **only** the isolated lab (Metasploitable) or **authorized** rooms — **never** a real account, website, or system.

If you find a real, exploitable bug

Do NOT exploit it, share it, or post it — even if you're "pretty sure" it's vulnerable.

DO report it privately to the owner. That's **responsible disclosure**, and companies even pay for it through **bug bounty** programs.

Report, don't exploit. That habit defines a professional.

Key vocabulary

Term	Meaning
Exploit	Code/steps that abuse a vulnerability
Public exploit	An exploit anyone can find and download
PoC	Code that <i>proves</i> a bug is real — sometimes rough/risky
CVE / NVD / CVSS	Bug ID / gov database / 0–10 severity score
Exploit-DB / searchsploit	Public exploit archive / CLI search tool
Banner	Text a service reveals, often its version
Payload	The part that does the attacker's goal (e.g., a shell)
Patching	Installing the update that removes the bug — the main defense



Lab launch

Platform: `searchsploit` / Exploit-DB on **Kali** vs. **Metasploitable 2** (or an authorized **TryHackMe** CVE room).

Your mission:

1. Re-confirm a service + version on the lab target.
2. Look it up by **CVE** on **NVD**; record the **CVSS** score.
3. Find a matching exploit with `searchsploit`.
4. **Read** it (complete the checklist), **adapt** it, **run** it on the lab box.
5. Document **CVE → exploit → result** and the **patch** that fixes it.

Confirm the lab is fully **isolated** before you touch anything.

Recap

- Workflow: **enumerate → version → CVE → matching exploit.**
- **NVD/CVSS** tell you how bad; **searchsploit/Exploit-DB** give you the code.
- **Match** carefully: service, version, platform, architecture.
- **Read before you run** — never trust untrusted code blindly.
- **Lightly adapt** (IP/port/parameter) only what's understood.
- The defense is **patching** and removing end-of-life software.

Exit ticket & discussion

Exit ticket: List three things you must understand about an exploit *before* running it.

Discussion: You find a public exploit for a CVE affecting software your school district uses, and you're "pretty sure" it's vulnerable. Walk through the responsible path. Where is the authorization line? What would **responsible disclosure** look like instead?